

PROPOSTA DE PROJETO FINAL

Programação Aplicada à Engenharia

1. Nome do projeto

CANTEIRO — jogo de encaixe de blocos com restrição de estabilidade estrutural.

2. Integrantes do grupo

Nome completo
Mateus Vitor Ferreira
Marcelo Camillo De Paula Leite
Wanessa Kylie Silva Medeiros

3. Resumo do projeto

CANTEIRO é um jogo de raciocínio espacial em que peças formadas por blocos descem por um tabuleiro e devem ser rotacionadas e encaixadas pelo jogador para formar linhas completas, que são então eliminadas. A proposta parte desse gênero clássico, mas introduz uma mecânica central inédita: cada peça representa um material de construção com massa específica própria (madeira, alvenaria, concreto e aço), e a estrutura empilhada é avaliada continuamente quanto ao seu equilíbrio.

A cada peça fixada, o programa recalcula o centro de massa do conjunto e o compara com a base de apoio disponível. Um indicador de estabilidade exibe esse desvio em tempo real. Se o jogador concentrar materiais pesados em um dos lados, o centro de massa sai da base e a estrutura entra em colapso: as camadas superiores desabam e são reorganizadas, penalizando a pontuação. Isso obriga o jogador a pensar não apenas em qual espaço vazio preencher, mas em como distribuir carga — transformando um jogo de encaixe em um exercício de intuição estrutural.

As regras do jogo serão escritas em Java e a interface rodará no navegador, em Svelte com TypeScript, tudo na máquina do jogador. Haverá tela de menu, seleção de dificuldade, partida, tela de colapso e ranking persistente de pontuações. Estão previstos ainda um modo "obra" com metas por fase e um sistema de repetição que permite rever a partida jogada por jogada.

4. Originalidade e diferenciais

- **Mecânica de centro de massa:** não se trata de um jogo de encaixe convencional com nova aparência, e sim de uma regra adicional que altera toda a estratégia da partida e conecta o jogo ao conteúdo de engenharia.
- **Materiais com comportamento distinto:** além da massa, cada material reage de forma própria ao ser fixado e ao ter sua linha eliminada, o que é resolvido por polimorfismo.
- **Sistema de repetição da partida:** todas as jogadas são registradas e podem ser reexecutadas passo a passo ao final.
- **Painel de análise:** ao fim da partida, o jogo exibe a evolução do índice de estabilidade e a distribuição de materiais utilizados.

5. Conceitos da disciplina aplicados

Conceito	Onde será aplicado no projeto
Pilha	Peça reservada pelo jogador e histórico de jogadas usado pelo sistema de repetição e pelo desfazer do modo treino.
Fila	Fila de próximas peças exibida ao jogador e fila de eventos do jogo (colapso, eliminação de linha, mudança de nível).
Lista	Representação do tabuleiro por linhas, conjunto de blocos de cada peça e lista de fragmentos gerados no colapso.
Tabela hash	Catálogo de materiais (código do material para suas propriedades físicas), tabela de rotações pré-calculadas e ranking de pontuações por jogador.
Herança	Classe abstrata Peca especializada nos sete formatos e classe abstrata Material especializada nos quatro tipos de construção.
Polimorfismo	Métodos de rotação, de pontuação e de reação ao encaixe reescritos em cada subclasse e chamados de forma uniforme pelo motor do jogo.
Encapsulamento e interfaces	Separação entre modelo, controle e interface; interfaces comuns para os observadores do estado e os elementos atualizáveis a cada ciclo.

6. Tecnologias e recursos

Linguagem: Java 17 no backend, com Javalin (HTTP e WebSocket), e TypeScript no frontend.

Interface: Svelte 5 no navegador, com o tabuleiro desenhado em Canvas; o estado da partida chega do backend por WebSocket.

Persistência: arquivos locais para ranking e repetições.

Testes: JUnit para colisão, rotação e centro de massa; Vitest para a interface.

Versionamento: Git e GitHub, com registro individual de contribuições.

Arquitetura: cliente-servidor local (127.0.0.1), com todas as regras no backend e a interface só desenhando o estado; entregue como um único JAR.

7. Objetivos

Objetivo geral: desenvolver o CANTEIRO, com as regras escritas em Java e a interface em Svelte 5 com TypeScript, em que a validade da estrutura construída depende do equilíbrio da sua distribuição de massa, aplicando de forma justificada os conceitos de programação orientada a objetos e de estruturas de dados da disciplina.

- **Hierarquias:** modelar as sete formas de peça e os quatro materiais em duas hierarquias de classes independentes.
- **Motor:** tratar queda, movimento, rotação com correção de posição, colisão e eliminação de linhas em tempo real.
- **Física:** implementar o cálculo incremental do centro de massa e a regra de colapso.
- **Estruturas de dados:** fila para as próximas peças e os comandos, pilha para a reserva e o histórico, lista para a composição das peças e tabela hash para o catálogo de materiais, o ranking e as sessões.
- **Comunicação:** expor o jogo por uma API REST e um canal WebSocket, com protocolo documentado e tipado dos dois lados.
- **Interface:** desenhar o tabuleiro em Canvas a 60 quadros por segundo, com indicação contínua do índice de estabilidade.

- **Persistência:** guardar ranking, configurações e repetições em arquivo.
- **Qualidade:** documentar as classes públicas com Javadoc, cobrir as regras críticas com testes e entregar tudo como um único JAR.

8. Escopo e público-alvo

O sistema é local e monousuário: o servidor escuta apenas em 127.0.0.1 e a interface abre no navegador da própria máquina, sem depender de internet. Destina-se a jogadores casuais a partir de doze anos e, em segundo plano, a estudantes de engenharia que queiram uma noção intuitiva de equilíbrio de cargas.

Fora do escopo: multijogador, ranking compartilhado pela internet, hospedagem em servidor público, adversário controlado por computador, edição de fases pelo usuário e versão para dispositivos móveis.

9. Regras de negócio

Código	Regra
RN01	O tabuleiro tem 10 colunas e 20 linhas visíveis, mais 2 linhas ocultas de geração no topo
RN02	Toda peça tem quatro blocos conexos, em uma das sete formas canônicas (I, O, T, S, Z, J, L)
RN03	O material da peça é sorteado entre os liberados na fase e define densidade, cor e multiplicador de pontos
RN04	As peças são geradas pelo método da sacola: as sete formas saem embaralhadas antes que qualquer uma se repita
RN05	O jogador pode reservar uma peça por vez, com uma troca a cada peça gerada
RN06	Uma linha completa é eliminada; eliminações simultâneas de 2, 3 ou 4 linhas recebem bônus progressivo
RN07	A massa de um bloco é a densidade do seu material; a massa da estrutura é a soma das massas dos blocos
RN08	O centro de massa horizontal é a média das colunas dos blocos, ponderada pela massa de cada um
RN09	O desvio é a distância entre o centro de massa e o eixo central da base, em colunas
RN10	O índice de estabilidade vai de 100 % (desvio nulo) a 0 % (desvio no limite do nível), linearmente
RN11	Desvio acima do limite causa colapso: os blocos acima da linha crítica se desprendem e caem por gravidade
RN12	Cada colapso desconta o equivalente a duas linhas eliminadas e incrementa o contador de colapsos
RN13	A partida termina se a peça nova já nascer colidindo ou se houver colapso com a pilha acima de 18 linhas
RN14	O nível sobe a cada 10 linhas; a queda fica mais rápida e o limite de desvio diminui
RN15	Só entram no ranking partidas concluídas sem uso do desfazer

10. Requisitos funcionais

E = essencial, presente na versão mínima entregável. D = desejável, implementado se houver prazo.

Código	Descrição	Prior.
RF01	Exibir um menu principal com nova partida, ranking, repetições, configurações e sair	E
RF02	Permitir escolher entre três dificuldades, que definem a velocidade inicial de queda e o limite de desvio	E
RF03	Gerar peças continuamente enquanto a partida estiver em andamento	E
RF04	Atribuir a cada peça um material sorteado entre os liberados na fase	E
RF05	Exibir as três próximas peças da fila, com forma e material	E
RF06	Descer a peça em queda uma linha a cada intervalo definido pelo nível	E
RF07	Mover a peça para a esquerda e para a direita, respeitando as bordas e os blocos fixados	E
RF08	Girar a peça nos dois sentidos, com deslocamento corretivo quando a rotação simples causar sobreposição	E
RF09	Permitir a queda instantânea da peça até a primeira posição de apoio	E
RF10	Permitir reservar a peça em queda e trocá-la pela reservada, uma vez por peça	E
RF11	Fixar a peça quando ela colidir com o fundo ou com um bloco fixado	E
RF12	Identificar e eliminar as linhas completas, descendo as linhas de cima	E
RF13	Calcular a pontuação considerando linhas simultâneas, material predominante e nível	E
RF14	Recalcular o centro de massa sempre que a composição do tabuleiro mudar	E
RF15	Exibir continuamente o índice de estabilidade, o desvio corrente e o limite tolerado	E
RF16	Sinalizar visualmente a aproximação do limite de desvio antes do colapso	E
RF17	Executar o colapso quando o desvio ultrapassar o limite, reacomodando os blocos desprendidos	E
RF18	Avançar de nível a cada dez linhas, ajustando velocidade e limite de desvio	E
RF19	Permitir pausar e retomar a partida	E
RF20	Encerrar a partida nas condições de fim de jogo e exibir a tela de resultado	E
RF21	Registrar a pontuação no ranking persistente, com o nome informado pelo jogador	E
RF22	Exibir o ranking com as dez melhores pontuações	E
RF23	Registrar em pilha todas as jogadas executadas na partida	E
RF24	Reproduzir passo a passo uma partida encerrada, a partir do histórico	D
RF25	Exibir, ao fim da partida, um relatório com a evolução do índice de estabilidade e a distribuição de materiais	D
RF26	Oferecer, no modo treino, o desfazer da última jogada	D

Código	Descrição	Prior.
RF27	Permitir configurar as teclas de comando	D
RF28	Tocar efeitos sonoros para fixação, eliminação de linha e colapso	D
RF29	Ao abrir o JAR, subir o servidor e abrir o jogo no navegador padrão	E
RF30	Pausar a partida automaticamente quando a conexão cair ou a aba perder o foco	E
RF31	Reconectar sozinho após uma queda de conexão e retomar a partida do ponto em que parou	D

11. Requisitos não funcionais

Código	Categoria	Descrição
RNF01	Desempenho	Desenhar o tabuleiro a 60 quadros por segundo e atualizar o estado 60 vezes por segundo, em máquina de dois núcleos e 4 GB de memória
RNF02	Desempenho	Recalcular o centro de massa de forma incremental, em tempo constante por bloco alterado
RNF03	Desempenho	No máximo 50 ms entre o pressionamento de uma tecla e a resposta visual
RNF04	Portabilidade	Rodar em Windows, Linux e macOS exigindo do jogador apenas Java 17 e um navegador atual
RNF05	Usabilidade	Comandos aprendidos sem manual, com legenda visível na tela de jogo
RNF06	Usabilidade	Materiais distinguíveis também por textura, atendendo jogadores com daltonismo
RNF07	Manutenibilidade	Javadoc completo nas classes e métodos públicos do backend; TSDoc nos tipos exportados do frontend
RNF08	Manutenibilidade	Regras do jogo inteiramente no modelo, sem dependência de servidor, JSON ou classes gráficas
RNF09	Manutenibilidade	Métodos com até 40 linhas, classes Java com até 400 e componentes Svelte com até 200
RNF10	Confiabilidade	Testes automatizados para colisão, rotação, eliminação de linhas, centro de massa e protocolo
RNF11	Confiabilidade	Falha na leitura de ranking ou configuração não impede o jogo; valores padrão são usados
RNF12	Segurança	Arquivos de dados validados na leitura; conteúdo malformado é rejeitado e registrado em log
RNF13	Segurança	Servidor escuta apenas em 127.0.0.1, nunca exposto na rede
RNF14	Confiabilidade	Toda mensagem recebida é validada; mensagem inválida gera erro, nunca derruba a partida
RNF15	Manutenibilidade	Protocolo entre backend e frontend documentado e tipado dos dois lados

Código	Categoria	Descrição
RNF16	Portabilidade	Um único JAR com o frontend embutido; o jogador não precisa de Node.js

12. Critérios de aceitação

Principais critérios que serão usados para considerar cada requisito atendido.

Req.	Critério	Verificação
RF03	Em 200 peças geradas, nenhuma forma se repete antes de as outras seis aparecerem	Teste automatizado
RF08	Toda peça encostada numa parede consegue girar, salvo quando os cinco deslocamentos falham	Teste automatizado
RF12	Eliminar quatro linhas de uma vez rende mais pontos que quatro eliminações isoladas	Teste automatizado
RF14	O centro de massa incremental coincide com o de varredura completa (tolerância 0,001)	Teste automatizado
RF17	Uma estrutura com aço numa só lateral entra em colapso em no máximo doze peças	Roteiro manual
RF21	As pontuações continuam no ranking depois de fechar e reabrir o jogo	Roteiro manual
RF29	Dois cliques no JAR abrem o jogo no navegador em até 3 segundos	Roteiro manual
RF30	Fechar a aba no meio da partida deixa a partida em pausa	Teste automatizado
RNF01	Mais de 55 quadros por segundo durante dez minutos de partida contínua	Medição em execução
RNF03	No máximo 50 ms entre a tecla e o redesenho	Medição em execução
RNF14	JSON malformado pelo WebSocket gera um erro e a partida continua	Teste automatizado
RNF16	O JAR roda numa máquina sem Node.js instalado	Roteiro manual

13. Cronograma

Semanas	Atividade	Produto
1 – 2	Requisitos, modelagem de classes e arquitetura	Especificação 1.0 (concluída)
3	Revisão da arquitetura; servidor Javalin; projeto Svelte; build único; protocolo	Especificação 2.1 e esqueleto ponta a ponta
3 – 4	Modelo: peças, materiais, tabuleiro e colisão, com testes	Núcleo testado, sem interface
5 – 6	Motor, rotação, linhas e pontuação; laço e WebSocket; tela de partida mínima	Jogável no navegador, sem estabilidade
7 – 8	Acumuladores, centro de massa, índice e colapso; painel de estabilidade	Mecânica diferencial completa

Semanas	Atividade	Produto
9 – 10	Telas completas, animações, texturas, retorno visual e reconexão	Interface integrada
11	Persistência, ranking, repetição, configurações e relatório	Requisitos desejáveis
12	Testes com jogadores, balanceamento, Javadoc e empacotamento	Entrega final

14. Riscos identificados

Risco	Probabilidade / impacto	Ação prevista
Colapso deixar o jogo frustrante ou injusto	Média / Alto	Testar com pessoas de fora do grupo na semana 8 e ajustar o limite de desvio
Parte da equipe sem experiência com Svelte e TypeScript	Alta / Alto	Frontend que só desenha o estado; componentes pequenos; exemplos no README; programação em par
Backend e frontend saírem de sincronia no protocolo	Média / Alto	Protocolo documentado, tipado dos dois lados, alterado no mesmo PR e conferido por teste
Latência do WebSocket deixar os controles lentos	Baixa / Médio	Comunicação só local, mensagens pequenas e medição desde o primeiro protótipo
Subestimar o esforço da rotação com deslocamento corretivo	Alta / Médio	Implementar primeiro a rotação simples e acrescentar os deslocamentos depois
Desequilíbrio na divisão do trabalho	Média / Alto	Revisão semanal do quadro de tarefas e registro individual de contribuições

15. Limitações reconhecidas

O modelo físico é uma simplificação deliberada: não há simulação de esforços internos, resistência dos materiais, deformação nem tombamento com rotação, e o colapso considera só o deslocamento horizontal do centro de massa. Isso basta para o jogo, mas não constitui análise estrutural no sentido da engenharia. O servidor foi feito para uso apenas local e não deve ser exposto na internet.

16. Divisão de responsabilidades

Previsão inicial, atualizada a cada etapa. Os testes automatizados são escritos por quem programa, no mesmo PR; roteiros manuais, aceitação e validação com jogadores ficam com a frente de testes. Cada PR é revisado por quem não o escreveu.

Frente de trabalho	Tipo	Responsável
Arquitetura, servidor e protocolo: Javalin, API REST, WebSocket e sessões de partida	Programação	Mateus Vitor Ferreira
Motor do jogo: tabuleiro, colisões, rotações, eliminação de linhas e pontuação	Programação	Mateus Vitor Ferreira
Física: materiais, centro de massa, índice de estabilidade e colapso	Programação	Mateus Vitor Ferreira
Tela de partida: tabuleiro em Canvas, painel de estabilidade, animações e teclado	Programação	Mateus Vitor Ferreira
Build e integração contínua: JAR único e testes automáticos em todo PR	Programação	Mateus Vitor Ferreira
Estruturas de dados: gerador por sacola, fila de próximas, pilha de reserva e histórico	Programação	Marcelo Camillo De Paula Leite
Persistência: materiais, configurações, ranking e repetições em arquivo	Programação	Marcelo Camillo De Paula Leite
Telas de apoio: menu, nova partida, fim de partida, ranking, repetições, relatório e configurações	Programação	Marcelo Camillo De Paula Leite
Requisitos: manter a especificação, a rastreabilidade e o status dos requisitos no README	Requisitos	Wanessa Kylie Silva Medeiros
Plano de testes: roteiros manuais a partir dos critérios de aceitação, execução a cada entrega e registro de defeitos	Testes	Wanessa Kylie Silva Medeiros
Validação com jogadores: sessões de teste, questionário e apoio ao balanceamento das dificuldades	Testes	Wanessa Kylie Silva Medeiros
Usabilidade e acessibilidade: revisão dos textos, legendas de teclas e texturas para daltonismo	Qualidade	Wanessa Kylie Silva Medeiros
Documentação e apresentação: relatório de plataforma e desvios, proposta, slides e ensaio das apresentações	Documentação	Wanessa Kylie Silva Medeiros